

Hybrid-INoT: гибридная агентная архитектура с внутренней ролевой сегрегацией для инженерных задач в условиях длинного общего контекста

Даниил Привезенцев^{1*}

^{1*}Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Внешняя мультиагентная оркестрация на базе больших языковых моделей повторно передаёт общий контекст отдельным ролям и тем самым увеличивает токенный расход. В работе предложена **Hybrid-INoT** — гибридная архитектура, в которой цикл *planner-worker-critic-self-check* выполняется внутри одного вызова модели, а внешними остаются инструментальные действия, верификация и компрессия контекста. Для архитектуры заданы операциональная модель, функция маршрутизации и аналитические условия токеной и экономической эффективности. Практическая проверка выполнена воспроизводимым стендом на HumanEval и контролируемом наборе задач. В реальном API-пилоте Gemini 3.1 Pro Preview ($n = 5$ задач на точку) Hybrid-INoT снизил средний токенный расход относительно Classical-MAS на 55.6–88.9% при росте целевого контекста от 256 до 16 384 токенов; в независимом прогоне Gemini 3.1 Flash-Lite ($n = 20$, seed 42) снижение составило 49.6–90.3%. В последнем прогоне парный выигрыш по U_{tok} статистически значим на всех четырёх длинах контекста после поправки Холма–Бонферрони ($p = 1.91 \cdot 10^{-6}$), однако pass@1 равен 1.0 у обеих архитектур, поэтому заявленный допуск на неухудшение качества статистически не доказан. Условие окупаемости малой модели подтверждено на пилоте HumanEval, но не подтверждено на используемом приближённом протоколе SWE-bench Lite. Аналитическая экстраполяция фактических расходов Pro даёт экономию API-инференса \$429.96 на 10 000 задач при равномерной смеси четырёх длин контекста (95% bootstrap CI \$267.06–\$617.17); это не оценка полного ТСО. Таким образом, данные поддерживают токенную

эффективность Hybrid-INoT в исследованном диапазоне, но не доказывают сохранение качества, перенос на реальные репозитории или экономию совокупной стоимости владения.

Keywords: Большие языковые модели, агентные системы, Introspection of Thought, токенизация эффективности, экономика инференса, HumanEval, воспроизводимый эксперимент

1 Введение

Современный агент для программной инженерии должен последовательно понять постановку задачи, извлечь релевантный контекст из репозитория и построить изменение, которое затем можно проверить тестами или статическим анализом. На практике эти функции часто распределяются между несколькими ролями, причём под *ролью* понимается отдельный инстанс (экземпляр) большой языковой модели или отдельный ИИ-агент с собственной инструкцией и отдельным обращением к API: одна роль строит план, другая предлагает решение, третья критикует промежуточный результат, четвёртая запускает внешнюю проверку. Таким образом, каждая роль — это самостоятельный вызов LLM, а не абстрактная функция внутри одного вызова. Такое разбиение удобно концептуально, но становится затратным, когда все роли вынуждены заново читать один и тот же длинный контекст.

В настоящей работе рассматривается именно этот сценарий: задачи, в которых общими для всех ролей оказываются фрагменты кода, документация, сообщения компилятора и журналы выполнения. Нас интересует не только итоговое качество решения, но и цена выбранной схемы оркестрации в токенах, времени ответа и денежной стоимости вызовов. Поэтому далее статья сопоставляет классическую внешнюю мультиагентную организацию с гибридной схемой, в которой ролевая дискуссия частично переносится внутрь одного вызова модели.

1.1 Проблема многократной передачи контекста и оценка её значимости

В агентных системах на основе больших языковых моделей типичной конфигурацией является внешняя мультиагентная оркестрация (Classical-MAS): отдельные процессы или вызовы реализуют роли планировщика, исполнителя, критика и тестировщика [12–14]. В задачах программной инженерии, где несколько ролей читают одни и те же длинные фрагменты кода, логов и документации, такая схема обладает структурным дефектом: *каждая роль получает копию общего контекста C_0 заново*, а между ролями дополнительно курсируют сообщения координации. Для количественной оценки этого эффекта введём следующие обозначения.

Обозначения к формуле ниже.

$\mathcal{A}$ — множество ролей в схеме Classical-MAS, каждая из которых реализована как отдельный вызов большой языковой модели; $|\mathcal{A}|$ — число таких ролей (мощность множества).

$\mathcal{C}_0$ — исходный общий контекст задачи (фрагменты кода, документация, журналы выполнения), одинаковый для всех ролей; $|\mathcal{C}_0|$ — его длина, измеряемая в токенах модели.

$\bar{h}$ — средняя длина одного координационного сообщения между ролями, также в токенах.

D_{ctx} — суммарный токенный расход на повторную передачу общего контекста всем ролям за одну задачу.

H_{coord} — суммарный токенный расход на координационные сообщения, которыми роли обмениваются в процессе решения задачи.

$\Theta(\cdot)$ — стандартная асимптотическая нотация: запись $f = \Theta(g)$ означает, что f растёт не медленнее и не быстрее g с точностью до постоянных множителей.

В терминах введённых обозначений токенный бюджет схемы Classical-MAS содержит слагаемое

$$D_{ctx} + H_{coord} \in \Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|) + \Theta(|\mathcal{A}|^2 \cdot \bar{h}). \quad (1)$$

Смысл формулы (1) следующий. Первое слагаемое $\Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|)$ описывает *линейный рост* затрат на контекст с числом ролей: если каждая из $|\mathcal{A}|$ ролей перечитывает исходный контекст длины $|\mathcal{C}_0|$, токенный расход на этот этап пропорционален произведению $|\mathcal{A}| \cdot |\mathcal{C}_0|$. Второе слагаемое $\Theta(|\mathcal{A}|^2 \cdot \bar{h})$ описывает *квадратичный рост* координационных затрат: в худшем случае каждая пара ролей может обмениваться сообщениями средней длины $\bar{h}$, и число таких пар порядка $|\mathcal{A}|^2$. Таким образом, при увеличении числа ролей затраты на передачу контекста растут линейно, а координационные затраты — квадратично, что делает схему Classical-MAS структурно неэффективной при большом $|\mathcal{A}|$ и длинном $|\mathcal{C}_0|$.

В выполненном API-пилоте этот структурный эффект наблюдался непосредственно. При целевом контексте 16 384 токена Classical-MAS расходовал в среднем 59 631 токен на задачу, из которых 57 485 (96.4%) приходились на вход, тогда как Hybrid-INoT расходовал 6 608 токенов. Стоимость одного случая составила соответственно \$0.14072 и \$0.02949 при зафиксированных ценах Gemini 3.1 Pro Preview [31]. Эти значения относятся к первым пяти задачам HumanEval и не должны переноситься на иной состав задач без повторного измерения.

Таким образом, актуальность проблемы определяется проверяемой величиной: в исследованном длинноконтекстном режиме повторная передача входа дала почти девятикратную разницу в суммарных токенах между B2 и B3. Далее теоретическая модель отделяется от эмпирического свидетельства, а область выводов ограничивается фактически выполненными пилотами.

1.2 Существующие подходы и их ограничения

Открытая литература содержит три родственных линии: (i) внутриконтекстная самокритика одной модели — Self-Refine [7], Reflexion [8]; (ii) чередование рассуждения и действия внутри одного агента — ReAct [6], SWE-agent [17]; (iii) внешняя

ролевая декомпозиция на отдельные вызовы — MetaGPT [12], ChatDev [13]. Ни одна из этих работ не задаёт *операционально отличимую* промежуточную конфигурацию, в которой (а) внутри одного вызова удерживается разделяемый контекст без дублирования, (б) ролевая сегрегация достаточна для критического разбора, (в) внешними остаются только операции с проверяемыми сигналами, такими как тесты, статический анализ и вызовы инструментов. Отсутствует также формальное условие, при котором такой гибрид доминирует над чисто внешней схемой Classical-MAS по ожидаемому токеному расходу.

1.3 Основные результаты и вклад статьи

В статье получены следующие результаты:

- C1.** Операциональное определение Hybrid-INoT с явной сигнатурой состояний, функцией маршрутизации `RouteMode` и условием сходимости внутреннего ролевого цикла (§4).
- C2.** Утверждение 2 о преимуществе Hybrid-INoT над Classical-MAS по ожидаемому токеному расходу при достаточно длинном общем контексте, доказанное через явный подсчёт токенов при выписанных предположениях (§5).
- C3.** Неравенство (16), задающее условие экономической эффективности малой модели самопроверки (§5).
- C4.** Система метрик токеной эффективности, стоимости и поддерживаемости кода с обоснованной формой агрегации (§6).
- C5.** Воспроизводимая эмпирическая проверка на реальных API-вызовах: масштабирование B2 и B3 по длине контекста, абляции, тест самопроверки и межмодельное сравнение E6 (§7).
- C6.** Раздельный вердикт по гипотезам: подтверждение токеного компонента H1 в прогоне Flash-Lite, частичная поддержка H2 только на HumanEval и отсутствие поддержки H3 в прямом сравнении архитектур (§7).

Исходный код стенда, конфигурации, сырые результаты и скрипт построения таблиц и графиков опубликованы в отдельном репозитории и зафиксированы по commit hash (§7).

2 Связанные работы и отличие предлагаемого подхода

2.1 От chain-of-thought к внутреннему ролевому дебату

В работе Wei et al. [5] показано, что явная вербализация промежуточных шагов рассуждения может повышать качество решения на задачах, требующих нескольких связанных логических переходов. В данном контексте под промежуточными шагами понимаются текстовые фрагменты, с помощью которых модель раскладывает сложный вывод на последовательность локальных подзадач, а под качеством — итоговая успешность решения целевой задачи. Подход Self-Refine [7] формализует цикл «генератор–обратная связь–исправление» внутри одной модели; Reflexion [8] дополняет такую схему механизмом рефлексивной памяти, а

ReAct [6] связывает внутреннее рассуждение с обращениями к внешним действиям и инструментам. Работа Sun и Zeng [1] описывает Introspection of Thought (INoT) как структурированный внутренний дебят нескольких ролей. В то же время Turpin et al. [9] показывают, что промежуточный текст цепочки не следует интерпретировать как достоверное описание скрытого вычислительного процесса. Поэтому в настоящей работе интроспекция рассматривается прежде всего как вычислительный механизм отбора кандидатов, критики и селективного перезапуска, а не как буквальное окно в скрытое состояние модели.

2.2 Мультиагентные системы для программной инженерии

Weiss [10] и Wooldridge [11] задают теоретическую рамку MAS. MetaGPT [12] и ChatDev [13] переносят её в контекст больших языковых моделей: роли реализуются как отдельные вызовы, каждая получает собственную копию контекста. Обзорные работы указывают на два ключевых ограничения Classical-MAS: распространение ошибки по цепочке и рост стоимости координации [14, 15]. SWE-bench и SWE-agent [16, 17] показывают, что практическая автоматизация задач программной инженерии требует длинного контекста и итеративного редактирования, что ещё сильнее обостряет проблему ретрансмиссии D_{ctx} .

2.3 Компрессия контекста и внешняя верификация

LongLLMLingua и LLMLingua-2 [20, 21] демонстрируют, что компрессия промпта сохраняет или улучшает качество на длинном контексте при снижении стоимости. Jia et al. [22] применяют этот подход к коду. Для самопроверки кода недостаточно языковой рефлексии: необходимы внешние наблюдаемые сигналы — тесты, статический анализ и сообщения компилятора [23–25].

2.4 Сопоставление Hybrid-INoT с предшественниками

Для систематизации различий между рассматриваемыми архитектурами выделим пять признаков: организацию контекста, структуру ролевого взаимодействия, способ проверки, механизм выбора режима работы и условие останова.

Таблица 1 показывает, что отдельные компоненты предлагаемой схемы встречаются в предшествующих работах, однако именно их сочетание в рамках единого внутреннего ролевого цикла, дополняемого внешней инструментальной верификацией и явной маршрутизацией, образует архитектуру Hybrid-INoT.

3 Формальная постановка задачи

3.1 Задача

Введём базовые множества, используемые в формальной постановке задачи. Их содержательный смысл — описать, с какими входными и выходными данными работает агент и в каком пространстве задана его случайность.

Таблица 1 Сравнение Hybrid-INoT с близкими архитектурными классами

Признак	Self-Refine / Reflexion	MetaGPT / ChatDev	Hybrid-INoT
Организация контекста	Один общий контекст без явного ролевого разделения.	Для каждой роли формируется отдельная копия общего контекста.	Один общий контекст сохраняется внутри вызова модели и разделяется между внутренними ролями.
Ролевое взаимодействие	Самокритика встроена в цикл одной модели.	Роли разведены по отдельным вызовам и обмениваются сообщениями.	Планирование, генерация и критика выполняются как внутренние роли в одном цикле.
Проверка решений	Преимущественно языковая самооценка.	Преимущественно ролевая языковая проверка; внешняя верификация зависит от реализации.	Итоговый кандидат проходит обязательную внешнюю инструментальную проверку; языковая критика используется как промежуточный фильтр.
Выбор режима работы	Фиксирован архитектурой метода.	Фиксирован архитектурой метода.	Задаётся явной функцией маршрутизации, переключающей внутренний и внешний режимы.
Условие остановки	Фиксированное число итераций или эпизодическое правило останова.	Завершение конвейера ролей или достижение согласия между ролями.	Остановка после успешной внешней проверки либо после достижения предельного числа итераций.

$\mathcal{X}$ — множество *текстовых спецификаций задач*. Каждый элемент $x \in \mathcal{X}$ — это строка естественного языка (например, сформулированная пользователем задача или формулировка задачи в HumanEval).

$\mathcal{C}$ — множество *контекстных артефактов*. Каждый элемент — это конечная последовательность текстовых фрагментов: файлов кода, кусков документации, сообщений компилятора, журналов выполнения. Конкретный контекст задачи $C_0 \in \mathcal{C}$ — это набор именно таких артефактов, имеющих отношение к задаче.

$\mathcal{Y}$ — множество *кандидатных решений*. Каждый элемент $y \in \mathcal{Y}$ — это текстовый артефакт, например исходный код функции или патч в формате **unified diff**, который затем подаётся во внешнюю проверяемую функцию (запуск тестов, статический анализ). Таким образом, y — не числовой вектор, а именно текст.

$(\Omega, \mathcal{F}, \mathbb{P})$ — *вероятностное пространство*, на котором определены все случайные величины задачи. Здесь Ω — множество возможных исходов одного запуска агента (все возможные сценарии развития процесса решения), $\mathcal{F}$ — σ -алгебра наблюдаемых событий над Ω (формальная структура, позволяющая корректно

говорить о вероятностях событий), $\mathbb{P}$ — вероятностная мера на $\mathcal{F}$, учитывающая сразу три источника случайности: стохастическую генерацию токенов в LLM при температуре > 0 , выбор начального числа генератора псевдослучайных чисел (*seed* — целое число, инициализирующее детерминированную последовательность псевдослучайных чисел; фиксация *seed* обеспечивает воспроизводимость запуска) и недетерминизм внешних инструментальных вызовов (например, таймауты тестов). Проще говоря, $(\Omega, \mathcal{F}, \mathbb{P})$ — это математический способ записать утверждение «запуск агента — это случайный эксперимент».

Определение 1 (Агентная инженерная задача). *Агентная инженерная задача — это кортеж $(x, C_0, \mathcal{Z}, \mathcal{V})$, где $x \in \mathcal{X}$ — текстовое описание задания; $C_0 \in \mathcal{C}$ — исходный контекст задачи, включающий код, документацию и журналы выполнения; $\mathcal{Z}$ — конечное множество разрешённых внешних инструментов (запуск тестов, линтер, компилятор, поиск по репозиторию и т. п.); $\mathcal{V}: \mathcal{Y} \rightarrow \{0, 1\}$ — внешняя проверяемая функция успеха, возвращающая 1 при корректном решении (например, при прохождении всего тестового набора) и 0 иначе.*

Случайные величины, индуцируемые стратегией агента.

Стратегия агента — это зафиксированная процедура, отображающая описание задачи x и контекст C_0 в распределение на $\mathcal{Y}$. Она индуцирует следующие случайные величины (функции из Ω в числовые множества):

$Y: \Omega \rightarrow \mathcal{Y}$ — *случайный выход агента*: итоговое кандидатное решение, которое агент возвращает в конце работы на конкретном запуске.

$T: \Omega \rightarrow \mathbb{R}_{\geq 0}$ — *суммарный токеновый расход* на одну задачу. Значение $T(\omega)$ — общее число токенов (суммарно по всем вызовам модели во всех ролях и итерациях), израсходованных на решение задачи в конкретном запуске ω . Здесь $\mathbb{R}_{\geq 0}$ — множество неотрицательных действительных чисел.

$C_{\text{inf}}: \Omega \rightarrow \mathbb{R}_{\geq 0}$ — *денежная стоимость инференса* на одну задачу (в валюте расчёта, например в долларах США), вычисляемая как произведение числа входных и выходных токенов на соответствующие тарифы API.

$\ell: \Omega \rightarrow \mathbb{R}_{\geq 0}$ — *задержка выполнения* одной задачи (wall-clock time в секундах) от получения запроса до возврата решения.

Случайность этих величин обусловлена стохастической генерацией LLM и недетерминизмом внешних инструментов; математическое ожидание $\mathbb{E}[\cdot]$ во всех последующих выражениях берётся по мере $\mathbb{P}$ на $(\Omega, \mathcal{F})$.

Целевая функция и ограничения.

Цель агента — максимизировать ожидаемое качество решения, удерживая ожидаемые затраты в пределах заданных бюджетов. Формально:

$$\max \mathbb{E}[\mathcal{V}(Y)] \quad \text{при условиях} \quad \mathbb{E}[T] \leq B, \quad \mathbb{E}[C_{\text{inf}}] \leq B_{\text{\$}}, \quad \mathbb{E}[\ell] \leq B_{\ell}. \quad (2)$$

Расшифровка всех величин в (2):

$\mathbb{E}[\mathcal{V}(Y)]$ — математическое ожидание функции успеха $\mathcal{V}$ от случайного выхода Y . Поскольку $\mathcal{V} \in \{0, 1\}$, эта величина равна вероятности того, что агент на случайном запуске вернёт корректное решение (то есть $\Pr[\mathcal{V}(Y) = 1]$).

$\mathbb{E}[T]$ — среднее число токенов, расходуемых агентом на одну задачу (по распределению, заданному $\mathbb{P}$).

$\mathbb{E}[C_{inf}]$ — средняя денежная стоимость инференса на одну задачу.

$\mathbb{E}[\ell]$ — средняя задержка ответа на одну задачу.

$B, B_{\$}, B_{\ell}$ — *верхние бюджетные лимиты*, заданные пользователем или эксплуатационным контекстом: B — допустимый средний токенный расход (в токенах), $B_{\$}$ — допустимая средняя стоимость (в валюте расчёта), B_{ℓ} — допустимая средняя задержка (в секундах).

Словами: формула (2) означает «среди стратегий, укладывающихся в заданные бюджеты по токенам, деньгам и времени, выбрать ту, у которой наибольшая ожидаемая доля успешных решений».

Сводка обозначений

Ниже собраны все обозначения, используемые в дальнейшем тексте, с явными определениями. Часть из них введена выше при постановке задачи; остальные появятся в разделах об архитектуре и аналитической модели.

$\mathcal{C}_0 \in \mathcal{C}$ — исходный общий контекст задачи (конечная последовательность текстовых артефактов); $|\mathcal{C}_0| \in \mathbb{N}$ — его длина в токенах модели.

$\hat{\mathcal{C}}$ — сжатая версия исходного контекста, используемая во внутреннем цикле Hybrid-INoT; $|\hat{\mathcal{C}}| \leq |\mathcal{C}_0|$.

$\mathcal{A}$ — конечное множество внешних ролей в схеме Classical-MAS, каждая из которых — отдельный вызов LLM; $|\mathcal{A}| \in \mathbb{N}$ — число таких ролей.

$Y: \Omega \rightarrow \mathcal{Y}$ — случайный выход агента; $y \in \mathcal{Y}$ — конкретный (реализованный) кандидат решения.

$K \in \mathbb{N}$ — число кандидатов, одновременно порождаемых ролью r_{work} на одной итерации внутреннего цикла (базовая конфигурация: $K = 3$).

$T: \Omega \rightarrow \mathbb{R}_{\geq 0}$ — случайная величина суммарного токенового расхода на одну задачу; $\mathbb{E}[T] = \int_{\Omega} T(\omega) d\mathbb{P}(\omega)$ — её математическое ожидание, вычисляемое по мере $\mathbb{P}$ на вероятностном пространстве $(\Omega, \mathcal{F}, \mathbb{P})$, введённом выше.

$U_{\text{tok}} = Q/T$ — показатель токеновой эффективности: отношение агрегированной оценки качества Q (см. §6) к суммарному токеновому расходу; единица измерения — «качество на 1000 токенов».

$\tau^* \in \mathbb{R}_{\geq 0}$ — пороговая длина исходного контекста (в токенах), начиная с которой Hybrid-INoT получает преимущество над Classical-MAS по ожидаемому токеновому расходу; её существование утверждается в §5, а эмпирическая оценка — в эксперименте ЕЗ (§7.2).

$\delta_{H1} \in (0, 1)$ — минимальный практически значимый относительный прирост показателя U_{tok} , используемый в гипотезе H1 (базовое значение $\delta_{H1} = 0,15$, то есть 15%).

$C_{check}^S \in \mathbb{R}_{\geq 0}$ — денежная стоимость одной предварительной проверки решения, выполняемой *малой* (дешёвой) моделью самопроверки; верхний индекс S обозначает small.

$C_{rerun}^L \in \mathbb{R}_{\geq 0}$ — денежная стоимость одного повторного запуска *крупной* (дорогой) модели; верхний индекс L обозначает large.

$\rho_0, \rho_1 \in [0, 1]$ — вероятности того, что потребуются дорогостоящий повторный запуск крупной модели: ρ_0 — без предварительной проверки малой моделью, ρ_1 — при наличии такой проверки; по построению $\rho_1 \leq \rho_0$.

$\mathbb{E}[\cdot]$ — математическое ожидание на вероятностном пространстве $(\Omega, \mathcal{F}, \mathbb{P})$.

$\mathbb{I}[\cdot]$ — индикаторная функция: $\mathbb{I}[A] = 1$, если условие A истинно, и 0 иначе.

3.2 Гипотезы исследования

Перед формулировкой гипотез поясним несколько обозначений и статистических процедур, встречающихся ниже, чтобы избавить читателя от необходимости искать их на стороне.

«п.п.» (*процентные пункты*) — единица разности двух величин, выраженных в процентах. Если одна конфигурация даёт $\text{pass}@1 = 68\%$, а другая = 66% , то разница составляет 2 процентных пункта ($68 - 66 = 2$), но при этом относительное изменение равно $\approx 3\%$. В тексте всегда используется «п.п.» именно в этом смысле: абсолютная разность процентов, а не отношение.

Парный критерий Вилкоксона (Wilcoxon signed-rank test) — непараметрический статистический тест для сравнения двух связанных выборок (например, значений метрики на одних и тех же задачах при двух конфигурациях). В отличие от t -критерия не требует нормальности распределения; проверяет гипотезу о симметрии распределения разностей относительно нуля. Применяется стандартно при небольших и негауссовских выборках [28].

Поправка Холма–Бонферрони (Holm–Bonferroni correction) — пошаговая процедура корректировки уровня значимости при множественном сравнении. Если одновременно проверяется m гипотез с p -значениями $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$, то k -я гипотеза отвергается, если $p_{(k)} \leq \alpha / (m - k + 1)$. Эта поправка контролирует вероятность хотя бы одной ложной находки (family-wise error rate) при $\alpha = 0,05$ и является менее консервативной, чем обычная поправка Бонферрони [29].

bootstrap-доверительный интервал — интервал для оценки параметра, полученный повторным ресэмплированием наблюдаемой выборки с возвращением (10^4 раз) и вычислением нужной статистики на каждом ресэмпле; квантили 2,5% и 97,5% распределения статистики дают 95%-й интервал.

seed (начальное число генератора псевдослучайных чисел) — целое число, которым инициализируется локальная псевдослучайная последовательность и передаётся поддерживающему этот параметр API. Фиксация seed обеспечивает повторяемость локальной выборки, но сама по себе не гарантирует битовую воспроизводимость закрытой LLM.

Synthetic Tool-Use Suite — контролируемый микротест стенда для сравнения последовательного и параллельного выполнения независимых инструментальных задержек. Он не является внешним бенчмарком и используется только как

проверка механизма НЗ; точная реализация находится в сопроводительном репозитории.

Н1 (*преимущество при длинном контексте*). Существует пороговая длина исходного контекста τ^* такая, что при $|C_0| > \tau^*$ архитектура Hybrid-INoT обеспечивает рост показателя U_{tok} не менее чем на $\delta_{H1} = 15\%$ по сравнению с Classical-MAS, при этом качество решения (pass@1) не ухудшается более чем на 2 п.п. В сравнении фиксируются модель, инструменты, проверка, задачи, seed, температура и промпты; варьируется схема оркестрации.

Проверка: масштабирование ЕЗ (§7.2) с парным критерием Вилкоксона, 95% bootstrap CI и поправкой Холма–Бонферрони.

Н2 (*экономическая оправданность малой модели самопроверки*). Существует режим, в котором предварительная проверка малой моделью уменьшает вероятность дорогостоящего повторного запуска настолько, что выполняется условие (16): $C_{\text{check}}^S < (\rho_0 - \rho_1) C_{\text{run}}^L$.

Проверка: пилот Е4 (§7.4) с прямыми измерениями на 20 задачах HumanEval и 20 случаях приближённого SWE-bench Lite.

НЗ (*граница применимости*). При задачах с истинно независимыми подзадачами и параллельными инструментами преимущество Hybrid-INoT по времени ответа исчезает или становится отрицательным.

Проверка: архитектурное сравнение на пяти задачах и отдельный контролируемый микротест последовательных/параллельных инструментов Е2с (§7.3).

Порог $\delta_{H1} = 15\%$ используется как минимально практически значимый эффект. Фактические оценки и статус каждой гипотезы приведены в табл. 5; наблюдаемая экономия не подменяет проверку полного составного критерия Н1.

4 Метод: Hybrid-INoT

4.1 Определение метода и базовая идея

Прежде чем переходить к формальному описанию, дадим содержательное определение метода, на котором строится предлагаемая архитектура, и сформулируем саму идею Hybrid-INoT в минимально необходимой форме.

Определение 2 (Introspection of Thought, INoT). Introspection of Thought (INoT) — это приём организации рассуждений большой языковой модели, при котором внутри одного обращения к модели разворачивается структурированный диалог нескольких ролей (ролевых префиксов: «планировщик», «исполнитель», «критик» и т. п.). Каждая роль читает и дописывает общий рабочий буфер в рамках одного прогона модели, что позволяет удерживать общий контекст без дублирования [1]. В отличие от внешней мультиагентной оркестрации, каждая роль INoT не является отдельным вызовом LLM — это ролевой префикс внутри единственного вызова.

Определение 3 (Hybrid-INoT). Hybrid-INoT — архитектура, предлагаемая в настоящей работе, которая комбинирует внутренний ролевой цикл planner–worker–critic в смысле Определения 2 с обязательной внешней инструментальной проверкой и предобработкой контекста. А именно:

- роли планирования (r_{plan}), генерации кандидатов (r_{work}) и критики (r_{crit}) выполняются как ролевые префиксы внутри одного вызова крупной LLM над сжатым общим контекстом $\hat{C}$;
- вовне вынесены три и только три класса операций: (i) компрессия контекста $C_0 \mapsto \hat{C}$ (отдельным лёгким модулем), (ii) внешняя проверяемая верификация r_{self} (запуск тестов, статического анализа, компилятора), (iii) функция маршрутизации **RouteMode**, решающая, какой режим работы применить к данной задаче.

Таким образом, Hybrid-INoT — это промежуточная конфигурация между чисто внутриконтекстной самокритикой (Self-Refine, Reflexion) и полностью внешней мультиагентной оркестрацией (MetaGPT, ChatDev), у которой внутри сохраняется разделяемый контекст и ролевая сегрегация, а снаружи остаются только операции с проверяемыми, инструментальными сигналами.

Дальнейшие подпункты формализуют эту идею: §4.2 — состояния и сигнатуры ролей, §4.3 — функция маршрутизации, §4.4 — внутренний цикл и условие его сходимости, §4.5 — компрессия контекста, §4.6 — селективный перезапуск.

4.2 Состояния и сигнатуры ролей

Определим состояние системы $\sigma = (x, \hat{C}, \mathcal{H}, i, b)$, где $x \in \mathcal{X}$ — текстовое описание задачи (см. Определение 1), $\hat{C}$ — сжатый общий контекст, $\mathcal{H}$ — история итераций (последовательность ранее рассмотренных планов, кандидатов и их оценок), $i \in \{0, \dots, I\}$ — счётчик итераций, $I \in \mathbb{N}$ — верхний предел числа итераций внутреннего цикла, $b \in \mathbb{N}$ — остаток токенового бюджета в токенах.

Пусть Π — пространство планов, то есть множество возможных текстовых планов решения (в базовой реализации — короткий маркированный список шагов на естественном языке). Пусть $K \in \mathbb{N}$ — число кандидатных решений, одновременно порождаемых ролью r_{work} на одной итерации внутреннего цикла; в базовой конфигурации $K = 3$. Тогда каждая роль задаётся сигатурой $r: \mathcal{S}_r^{\text{in}} \rightarrow \mathcal{S}_r^{\text{out}}$:

$$r_{\text{plan}}: (x, \hat{C}, b) \mapsto \pi \in \Pi, \quad (3)$$

$$r_{\text{work}}: (\pi, \hat{C}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \quad (4)$$

$$r_{\text{crit}}: (\mathbf{y}, \hat{C}) \mapsto (s_1, \dots, s_K) \in [0, 1]^K, \quad (5)$$

$$r_{\text{self}}: (y \in \mathcal{Y}) \mapsto (v, M) \in \{0, 1\} \times [0, 1]. \quad (6)$$

Расшифровка сигнатур (3)–(6):

r_{plan} — роль *планировщика*. Входы: текст задачи x , сжатый контекст $\hat{C}$, остаток бюджета b . Выход: план $\pi \in \Pi$ — последовательность шагов решения.

r_{work} — роль *исполнителя* (генератора кандидатов). Входы: план π , сжатый контекст $\hat{C}$, число требуемых кандидатов K . Выход: набор $\mathbf{y} = (y_1, \dots, y_K)$ из K кандидатных решений $y_k \in \mathcal{Y}$; запись $\mathcal{Y}^K$ — K -я декартова степень множества $\mathcal{Y}$. Значение K задаётся гиперпараметром архитектуры (в базовой конфигурации $K = 3$) и регулирует разнообразие: при $K = 1$ критик всегда оценивает единственный вариант и роль r_{crit} вырождается; увеличение K повышает вероятность получить хоть одно корректное решение ценой пропорционального роста выходных токенов.

r_{crit} — роль *критика*. Входы: набор кандидатов $\mathbf{y}$ и контекст $\hat{C}$. Выход: набор скалярных оценок $(s_1, \dots, s_K) \in [0, 1]^K$, где s_k — оценка качества k -го кандидата (1 — безусловно принять, 0 — безусловно отвергнуть).

r_{self} — *внешний процесс самопроверки*: запуск тестов, статического анализа и линтера. Вход: один отобранный кандидат y . Выход: пара (v, M) , где $v \in \{0, 1\}$ — бинарный вердикт внешней проверки (1 — тесты и проверки пройдены, 0 — нет), а $M \in [0, 1]$ — скалярная оценка поддерживаемости кода (см. формулу (21) в §6).

Роли $r_{plan}, r_{work}, r_{crit}$ реализуются как ролевые префиксы внутри *одного* вызова крупной модели над общим $\hat{C}$; роль r_{self} — отдельный внешний процесс (запуск тестов, линтера, статического анализатора). Содержательный смысл сигнатур: внутренний цикл вырабатывает план, по нему генерирует K кандидатов, оценивает их критиком и передаёт лучшего на внешнюю инструментальную проверку, которая возвращает единственный объективный сигнал об успехе.

4.3 Функция маршрутизации

Содержательный смысл.

`RouteMode` — это *функция выбора режима работы* Hybrid-INoT для данной задачи. Она отвечает на вопрос: решать задачу внутренним ролевым циклом в рамках одного вызова модели (`internal`) или отдать задачу внешней мультиагентной оркестрации (`external`), поскольку в данной постановке внутренний режим заведомо неэффективен? Решение принимается на основе трёх дешёвых признаков задачи: требуется ли обязательный внешний инструмент, доминирует ли параллелизм подзадач и достаточно ли длинный контекст. Формальное определение — ниже.

Определение 4 (`RouteMode`). Пусть $\varphi_{\text{tool}}(x) \in \{0, 1\}$ указывает на обязательность внешнего инструмента, $\varphi_{\text{par}}(x) \in \{0, 1\}$ — на доминирующий параллелизм подзадач, $|\hat{C}|$ — длина сжатого контекста. Тогда

$$\text{RouteMode}(x, \hat{C}) = \begin{cases} \text{external}, & \varphi_{\text{tool}}(x) = 1 \vee \varphi_{\text{par}}(x) = 1 \vee |\hat{C}| < \tau_{\text{route}}^*, \\ \text{internal}, & \text{иначе.} \end{cases}$$

Параметры $\varphi_{\text{tool}}, \varphi_{\text{par}}$ определяются отдельным классификатором над x (в базовой реализации — лёгким классификатором на основе инструкции к модели);

порог $\tau_{\text{route}}^* = 1024$ токена остаётся инженерной настройкой. Его нельзя отождествлять с цензурированным исследовательским порогом $\tau^* \leq 256$, полученным в ЕЗ.

4.4 Внутренний цикл и условие сходимости

Algorithm 1 Hybrid-INoT: внутренний интроспективный цикл

Require: задача x , контекст $\mathcal{C}_0$, бюджет B , максимум итераций I , число кандидатов K

Ensure: решение y^* , флаг v^* , оценка M^*

```

1:  $\mathcal{C} \leftarrow \text{PolicySanitize}(x, \mathcal{C}_0)$ 
2:  $\hat{\mathcal{C}} \leftarrow \text{CompressIfNeeded}(\mathcal{C})$  ▷ Алг. 2
3:  $m \leftarrow \text{RouteMode}(x, \hat{\mathcal{C}})$  ▷ Опр. 4
4: if  $m = \text{external}$  then return  $\text{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$ 
5: end if
6:  $s^{\text{best}} \leftarrow -\infty$ ;  $y^{\text{best}} \leftarrow \perp$ 
7: for  $i = 1$  to  $I$  do
8:    $\pi_i \leftarrow r_{\text{plan}}(x, \hat{\mathcal{C}}, B - \text{spent})$ 
9:    $\{y_{i,k}\} \leftarrow r_{\text{work}}(\pi_i, \hat{\mathcal{C}}, K)$ 
10:   $(s_{i,1}, \dots, s_{i,K}) \leftarrow r_{\text{crit}}(\{y_{i,k}\}, \hat{\mathcal{C}})$ 
11:   $k^* \leftarrow \arg \max_k s_{i,k}$ ;  $y_i \leftarrow y_{i,k^*}$ 
12:  if  $s_{i,k^*} > s^{\text{best}}$  then  $s^{\text{best}} \leftarrow s_{i,k^*}$ ;  $y^{\text{best}} \leftarrow y_i$ 
13:  end if
14:   $(v_i, M_i) \leftarrow r_{\text{self}}(y_i)$  ▷ внешние тесты и статический анализ
15:  if  $v_i = 1$  then return  $(y_i, 1, M_i)$ 
16:  end if
17:   $\hat{\mathcal{C}} \leftarrow \text{UpdateContextWithFailures}(\hat{\mathcal{C}}, v_i)$ 
18:  if  $\text{stagnation}(s_{i,k^*} - s_{i-1,k^*}) < \varepsilon$  then break ▷ см. УТВ. 1
19:  end if
20: end for
21: return  $(y^{\text{best}}, 0, M(y^{\text{best}}))$ 

```

Утверждение 1 (Терминация внутреннего цикла). Пусть каждая операция внутри одной итерации алгоритма 1 завершается за конечное время, а число итераций ограничено сверху параметром I . Тогда алгоритм завершится не позднее чем после I итераций: он либо вернёт решение сразу после успешной внешней проверки $v_i = 1$, либо остановится раньше по правилу $\text{stagnation}(s_{i,k^*} - s_{i-1,k^*}) < \varepsilon$, либо после исчерпания лимита итераций вернёт лучший найденный кандидат.

Доказательство Внешний цикл по индексу i содержит не более I проходов. Каждая итерация включает конечный набор шагов: построение плана, генерацию K кандидатов,

критическую оценку, внешнюю проверку и обновление контекста. По предположению все эти операции завершаются за конечное время, следовательно, завершается и любая отдельная итерация. Поэтому алгоритм либо прекращается досрочно по инструкции **return** или **break**, либо после I -й итерации достигает последней инструкции **return**. Тем самым время работы внутреннего цикла сверху ограничено конечным числом итераций, а утверждение доказано. $\square$

Замечание

Утверждение 1 гарантирует только завершение алгоритма, но не монотонный рост оценок s_i и не корректность самого критика. Сценарий, в котором критик систематически подтверждает неверные решения, рассматривается отдельно в E2b (§7.3) как одна из угроз валидности.

4.5 Компрессия контекста

Algorithm 2 Компрессия длинного контекста

Require: $\mathcal{C}_0$, порог длины τ_{len}

Ensure: $\hat{\mathcal{C}}$

- 1: **if** $|\mathcal{C}_0| \leq \tau_{\text{len}}$ **then return** $\mathcal{C}_0$
 - 2: **end if**
 - 3: разбить $\mathcal{C}_0$ на смысловые блоки $\{b_j\}$
 - 4: оценить $\text{rel}(b_j)$ с помощью классификатора по типу LLMlingua-2 [21]
 - 5: удалить дубликаты, низкоинформативные логи, вторичную документацию
 - 6: сохранить API-сигнатуры, трассы с ошибками, изменяемые файлы и ограничения задачи
 - 7: **return** $\hat{\mathcal{C}}$ с $|\hat{\mathcal{C}}| \leq \tau_{\text{len}}$
-

4.6 Селективный перезапуск

Содержательный смысл.

Селективный перезапуск — механизм экономии токенов и денежного бюджета при неудачной попытке решения. Вместо того чтобы при отказе критика перезапускать всю итерацию с нуля или вообще прекращать работу, процедура отбирает $M \leq K$ лучших по оценке критика кандидатов, поочерёдно прогоняет их через внешнюю верификацию **ExternalVerify** и, если ни один не прошёл, формирует новый промпт *только* на основе *подтверждённых* инструментами сигналов ошибки (сообщений тестов, трасс исключений, вывода статического анализатора), а не субъективных комментариев критика. Это ограничивает распространение языковых галлюцинаций между итерациями и уменьшает число дорогих повторных запусков. Алгоритм 3 формализует процедуру.

Algorithm 3 Селективный перезапуск по подтверждённым сигналам

Require: кандидаты $\{y_1, \dots, y_K\}$, оценки $\{s_k\}$, бюджет повторов $R \in \mathbb{N}_0$

Ensure: итоговое решение y^*

```
1: отсортировать  $\{y_k\}$  по убыванию  $s_k$ ; оставить верхние  $M \leq K$ 
2: for каждый  $y_m$  из отобранных do
3:    $q_m \leftarrow \text{ExternalVerify}(y_m)$   $\triangleright$  запуск тестов и статического анализа
4:   if  $q_m = 1$  then return  $y_m$ 
5:   end if
6: end for
7: if  $R > 0$  then
8:   сформировать новый промпт, включив только подтверждённые инстру-
     ментами сигналы ошибки
9:   return  $\text{SelectiveRerun}(R - 1)$ 
10: end if
11: return  $\arg \max_k s_k$   $\triangleright$  если бюджет перезапусков исчерпан, вернуть лучший
    по критику
```

Расшифровка входов и выходов Алгоритма 3.

$\{y_1, \dots, y_K\} \subset \mathcal{Y}$ — набор кандидатных решений, полученных от роли r_{work} на текущей итерации.

$\{s_1, \dots, s_K\} \subset [0, 1]$ — соответствующие оценки от роли-критика r_{crit} .

$M \in \{1, \dots, K\}$ — число кандидатов, допускаемых к *дорогой* внешней проверке (в базовой конфигурации $M = \min(K, 3)$).

$R \in \mathbb{N}_0$ — оставшийся бюджет рекурсивных перезапусков; $\mathbb{N}_0 = \{0, 1, 2, \dots\}$. При $R = 0$ дальнейшие перезапуски запрещены.

$\text{ExternalVerify}(y_m) \in \{0, 1\}$ — внешняя проверка (вызов тестового раннера, линтера, статического анализатора); возвращает 1 только при полном успехе.

$y^* \in \mathcal{Y}$ — итоговое возвращаемое решение: либо первый прошедший внешнюю проверку кандидат, либо лучший по оценке критика при исчерпании бюджета R .

Почему именно подтверждённые сигналы.

Восстанавливая промпт только из объективных сообщений инструментов, алгоритм обрывает обратную связь «критик подтверждает собственную ошибку $\rightarrow$ следующее поколение наследует её», которая систематически ухудшает сходимость в чисто языковых Self-Refine/Reflexion-циклах [7, 8].

5 Аналитическая модель токенных и стоимостных затрат

5.1 Структура аналитических соотношений

Анализ в данном разделе включает два типа соотношений. К первому относятся прямые балансовые равенства и неравенства, задающие структуру токенных и денежных затрат. Ко второму относятся утверждения о сравнении архитектур

при фиксированных предположениях относительно длины контекста, стоимости координации и внутренней критики. Такое разделение позволяет отличить общие расчётные соотношения от результатов, требующих дополнительных условий.

5.2 Токенные бюджеты Classical-MAS и Hybrid-INoT

Перед формулировкой утверждения о преимуществе по токенам введём обозначения для *всех* слагаемых, из которых складываются бюджеты T_{MAS} и T_{INoT} , и зафиксируем соответствующие предположения. Все величины далее выражены в токенах модели и относятся к одной задаче.

T_{MAS} — суммарный токенный расход агента Classical-MAS на одну задачу (случайная величина по распределению $\mathbb{P}$).

T_{INoT} — суммарный токенный расход агента Hybrid-INoT на одну задачу.

$T_{\text{ctx}}^{\text{MAS}}$ — вклад повторной передачи общего контекста в бюджет Classical-MAS.

$T_{\text{coord}}^{\text{MAS}}$ — вклад координационных сообщений между ролями в бюджет Classical-MAS.

T_{gen} — совокупный вклад собственно генерации кандидатов и прочих членов, не зависящих линейно от числа внешних ролей $|\mathcal{A}|$ (одинаково присутствующий в обеих архитектурах и потому сокращающийся при вычитании).

T_{cmp} — стоимость однократной компрессии контекста $\mathcal{C}_0 \mapsto \hat{\mathcal{C}}$.

$T_{\text{core}}^{\text{add}}$ — дополнительная стоимость внутренней критики в Hybrid-INoT сверх обычной генерации (расход на ролевые префиксы и развёрнутый внутренний дебат).

T_{check} — суммарная стоимость внешних верификаций (запусков тестов/линтера, передачи их вывода обратно в модель) за одну задачу.

T_{rerun} — случайная величина стоимости повторных запусков при провале внешней верификации.

$\alpha \in (0, 1]$ — доля ролей Classical-MAS, читающих полный контекст (см. Предположение 1).

$\gamma \in [0, \alpha|\mathcal{A}|)$ — коэффициент, ограничивающий стоимость компрессии: $T_{\text{cmp}} \leq \gamma|\mathcal{C}_0|$.

$\beta \in [0, \alpha|\mathcal{A}| - \gamma)$ — коэффициент, ограничивающий дополнительную стоимость критики: $T_{\text{core}}^{\text{add}} \leq \beta|\hat{\mathcal{C}}|$.

$I \in \mathbb{N}$ — верхний предел числа итераций в обеих архитектурах.

$\bar{h}$ — средняя длина координационного сообщения между ролями (в токенах).

Неравенства между α, β, γ означают, что стоимость компрессии и стоимость внутренней критики *вместе* всё же меньше, чем объём повторно передаваемого контекста всеми ролями Classical-MAS; в противном случае выигрыш структурно невозможен. Эти ограничения подлежат эмпирической проверке при конкретной реализации.

Предположение 1. Среди ролей $\mathcal{A}$ в схеме Classical-MAS не менее $\alpha|\mathcal{A}|$ ролей, $\alpha \in (0, 1]$, читают общий контекст $\mathcal{C}_0$ целиком на каждой итерации. Это эмпирическое наблюдение для существующих реализаций MetaGPT [12] и

ChatDev [13], в которых планировщик, исполнитель и критик получают полную копию контекста для корректной работы.

Предположение 2. Длина координационных сообщений ограничена $\bar{h}$ токенами, число итераций в обеих схемах равно I , стоимость компрессии $T_{cmp} \leq \gamma|C_0|$ с $\gamma < \alpha|\mathcal{A}|$, стоимость внутренней критики $T_{core}^{add} \leq \beta|\hat{C}|$ с $\beta < \alpha|\mathcal{A}| - \gamma$. Верхние оценки γ, β согласуются с эмпирикой работ по компрессии промпта (*LongLLMLingua*, *LLMLingua-2*) [20, 21], где коэффициенты сжатия составляют 3–20×, то есть $\gamma \leq 0,3$.

Утверждение 2 (Токенное преимущество при длинном контексте). При Предположениях 1–2 и $|\mathcal{A}| \geq 3$ существует порог $\tau^* > 0$ такой, что при $|C_0| > \tau^*$ выполняется

$$\mathbb{E}[T_{MAS}] - \mathbb{E}[T_{INoT}] \geq (\alpha|\mathcal{A}| - \gamma - \beta) \cdot |C_0| + \Theta(|\mathcal{A}|^2 \bar{h}) - T_{check} - \mathbb{E}[T_{rerun}] > 0. \quad (7)$$

Расшифровка (7). В левой части — разность *ожидаемых* (то есть усреднённых по случайности запуска) токенных бюджетов двух архитектур. В правой части последовательно: линейное по $|C_0|$ слагаемое с положительным коэффициентом $\alpha|\mathcal{A}| - \gamma - \beta > 0$ (собственно экономия от однократной передачи сжатого контекста вместо повторной передачи полного); добавочный член $\Theta(|\mathcal{A}|^2 \bar{h})$, отражающий сэкономленные координационные сообщения; и два *штрафных* члена T_{check} и $\mathbb{E}[T_{rerun}]$ — дополнительные расходы Hybrid-INoT на внешнюю верификацию и возможные повторные запуски. Преимущество существует, когда линейный выигрыш по контексту при $|C_0| > \tau^*$ перекрывает эти штрафы.

Вывод из предположений Разложим токенный бюджет Classical-MAS на три слагаемых:

$$T_{MAS} = T_{ctx}^{MAS} + T_{coord}^{MAS} + T_{gen}. \quad (8)$$

По Предположению 1 на каждой из I итераций не менее $\alpha|\mathcal{A}|$ ролей перечитывают полный контекст C_0 , поэтому

$$T_{ctx}^{MAS} \geq I \cdot \alpha|\mathcal{A}| \cdot |C_0|. \quad (9)$$

По Предположению 2 координационные сообщения между парами ролей (число таких пар в худшем случае $\Theta(|\mathcal{A}|^2)$) дают $T_{coord}^{MAS} \in \Theta(I \cdot |\mathcal{A}|^2 \bar{h})$. Подставляя в (8) и беря математическое ожидание, получаем

$$\mathbb{E}[T_{MAS}] \geq I \cdot \alpha|\mathcal{A}| \cdot |C_0| + \Theta(I \cdot |\mathcal{A}|^2 \bar{h}) + \mathbb{E}[T_{gen}]. \quad (10)$$

Для Hybrid-INoT контекст сжимается однократно ($T_{cmp} \leq \gamma|C_0|$), внутри одного вызова все роли делят общий $\hat{C}$ без дублирования, дополнительная стоимость критики ограничена $T_{core}^{add} \leq \beta|\hat{C}| \leq \beta|C_0|$, а внешняя часть даёт T_{check} и $\mathbb{E}[T_{rerun}]$. Отсюда

$$\mathbb{E}[T_{INoT}] \leq I \cdot |\hat{C}| + T_{cmp} + T_{core}^{add} + T_{check} + \mathbb{E}[T_{rerun}] + \mathbb{E}[T_{gen}]. \quad (11)$$

Заметим, что $I \cdot |\hat{C}| \leq I \cdot |C_0|$; в пределе сильной компрессии этот член может оказаться существенно меньше $I \cdot \alpha|\mathcal{A}| \cdot |C_0|$ из (9). Вычитая (11) из (10), общий член $\mathbb{E}[T_{gen}]$ сокращается, и мы получаем

$$\mathbb{E}[T_{MAS}] - \mathbb{E}[T_{INoT}] \geq (I\alpha|\mathcal{A}| - I - \gamma - \beta) \cdot |C_0| + \Theta(I|\mathcal{A}|^2 \bar{h}) - T_{check} - \mathbb{E}[T_{rerun}]. \quad (12)$$

При фиксированных $I, \alpha, |\mathcal{A}|, \gamma, \beta$ коэффициент при $|\mathcal{C}_0|$ положителен (так как $I\alpha|\mathcal{A}| > \gamma + \beta + I$ при $|\mathcal{A}| \geq 3$ и α, β, γ из Предположения 2). Обозначим этот коэффициент $A > 0$, а отрицательные константы — $B' = T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}]$. Тогда правая часть (12) равна $A \cdot |\mathcal{C}_0| + \Theta(I|\mathcal{A}|^2\bar{h}) - B'$, и она становится положительной при

$$|\mathcal{C}_0| > \frac{B' - \Theta(I|\mathcal{A}|^2\bar{h})}{A} =: \tau^*,$$

что и даёт существование искомого порога. Поглощая коэффициент I в переопределение α (без потери общности), получаем формулировку (7). $\square$

Область применимости утверждения.

Утверждение 2 задаёт нижнюю оценку разности ожидаемых токеновых бюджетов при Предположениях 1–2. При $|\mathcal{A}| = 2$ (например, plan+exes без выделенного критика) знак этой разности может изменяться. Аналогично, при $|\mathcal{C}_0| < \tau^*$ преимущество Hybrid-INoT не гарантируется. Значение τ^* зависит от $\alpha, \beta, \gamma, |\mathcal{A}|, I$ и должно уточняться эмпирически (эксперимент E3, §7.2).

5.3 Верхняя граница токенового расхода

Введём величины, входящие в утверждение о верхней границе:

$T_{\text{INoT}}^{\text{worst}}$ — верхняя оценка токенового расхода Hybrid-INoT в худшем случае, то есть когда все I итераций внутреннего цикла заканчиваются отрицательным вердиктом внешней проверки ($v_i = 0$) и алгоритм 1 доходит до полного исчерпания бюджета итераций.

p^L — число входных (prompt) токенов крупной модели на один вызов (*per one candidate generation*); верхний индекс L обозначает large.

r^L — число выходных (response) токенов крупной модели на один вызов.

$T_{\text{rerun}}^{\text{unit}}$ — стоимость одного повторного запуска (включая повторную передачу истории и вывода инструментов).

Утверждение 3 (Верхняя граница расхода токенов Hybrid-INoT). *В худшем случае, когда все I итераций заканчиваются $v_i = 0$,*

$$T_{\text{INoT}}^{\text{worst}} \leq T_{\text{cmp}} + I \cdot K \cdot (p^L + r^L) + I \cdot T_{\text{check}} + I \cdot T_{\text{rerun}}^{\text{unit}}. \quad (13)$$

Вывод оценки Токенный расход каждой из I итераций алгоритма 1 складывается из четырёх независимых составляющих:

1. *Компрессия контекста* — выполняется *один раз* перед началом цикла, вклад ровно T_{cmp} (слагаемое без множителя I).
2. *Генерация K кандидатов ролью r_{work}* — каждая генерация требует $p^L + r^L$ токенов крупной модели (входные плюс выходные), всего $K \cdot (p^L + r^L)$ на итерацию. За I итераций набегает $I \cdot K \cdot (p^L + r^L)$. Слагаемые планирования r_{plan} и критики r_{crit} поглощены в эту же оценку верхней границей (их вклад ограничен в рамках одного вызова, разделяющего контекст).
3. *Внешняя верификация* — на каждой итерации вклад не превосходит T_{check} ; за I итераций даёт $I \cdot T_{\text{check}}$.

4. *Повторные запуски* — в худшем случае на каждой итерации возникает один повторный запуск стоимостью $T_{\text{rerun}}^{\text{unit}}$, что за I итераций даёт $I \cdot T_{\text{rerun}}^{\text{unit}}$.

Сложив четыре составляющих, получаем (13). $\square$

Полученная оценка используется как верхнее ограничение при конечном бюджете B : при $T \rightarrow B$ алгоритм 1 досрочно возвращает лучшего из найденных кандидатов ($\arg \max_k s_k$), что гарантирует соблюдение бюджета даже в наихудшем сценарии.

5.4 Условие экономической эффективности малой модели самопроверки

Обозначения для формулы стоимости инференса.

Пусть агент в процессе решения одной задачи делает несколько вызовов крупной (large, верхний индекс L) и малой (small, верхний индекс S) моделей с индексом i , пробегающим по вызовам. Для каждого вызова обозначим:

p_i^L, r_i^L — число входных (prompt) и выходных (response) токенов i -го вызова крупной модели.

p_i^S, r_i^S — то же для i -го вызова малой модели.

$c_L^{\text{in}}, c_L^{\text{out}}$ — цены *крупной* модели за один входной и выходной токен соответственно (в валюте расчёта, обычно долл. США за токен; у провайдеров API обычно указывается цена за 10^6 токенов, и значения пересчитываются делением на 10^6).

$c_S^{\text{in}}, c_S^{\text{out}}$ — то же для малой модели.

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \quad (14)$$

Формула (14) — прямая балансовая запись: денежная стоимость инференса на одну задачу равна сумме по всем вызовам произведений числа токенов на соответствующий тариф, отдельно для входных/выходных токенов и крупной/малой моделей.

Обозначения для условия эффективности.

Рассмотрим схему, в которой после получения кандидата y крупной моделью он предварительно проверяется *малой* моделью; если малая модель сигнализирует об ошибке, запускается повторная генерация крупной моделью. Введём:

C_{check}^S — денежная стоимость одной предварительной проверки малой моделью (включает входной и выходной токанный расход проверочного вызова, пересчитанный в деньги по (14)).

C_{rerun}^L — денежная стоимость одного повторного запуска крупной модели (при провале текущего кандидата).

ρ_0 — вероятность того, что потребуются дорогостоящий повторный запуск в *базовой* схеме без *малой* модели *самопроверки* (то есть доля ошибочных кандидатов, не выловленных ни одним дешёвым механизмом).

ρ_1 — та же вероятность *при наличии* предварительной проверки малой моделью; по построению $0 \leq \rho_1 \leq \rho_0 \leq 1$, причём часть ошибочных кандидатов малая модель отсекает, прежде чем они попадут в крупную.

Вывод условия.

Сравним ожидаемые расходы на одну задачу в двух сценариях.

Сценарий 0 (без малой модели): расход = 0 при успехе; с вероятностью ρ_0 возникает повторный запуск крупной модели стоимостью C_{rerun}^L . Ожидаемые дополнительные расходы: $\mathbb{E}_0 = \rho_0 \cdot C_{\text{rerun}}^L$.

Сценарий 1 (с малой моделью): *всегда* платим C_{check}^S за предварительную проверку; дополнительно с вероятностью ρ_1 всё ещё нужен повторный запуск стоимостью C_{rerun}^L . Ожидаемые дополнительные расходы: $\mathbb{E}_1 = C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L$.

Малая модель экономически эффективна тогда и только тогда, когда $\mathbb{E}_1 < \mathbb{E}_0$, то есть

$$C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L < \rho_0 \cdot C_{\text{rerun}}^L, \quad (15)$$

что после переноса $\rho_1 \cdot C_{\text{rerun}}^L$ в правую часть даёт

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \quad (16)$$

Содержательная расшифровка (16). Левая часть — фиксированная стоимость предварительной проверки на каждой задаче. Правая часть — ожидаемая экономия: разность вероятностей $\rho_0 - \rho_1 \geq 0$ показывает, какую долю дорогих повторов удаётся предотвратить благодаря малой модели; умножив её на стоимость одного повторного запуска C_{rerun}^L , получаем ожидаемую денежную экономию. Неравенство выполняется, когда фиксированный взнос в предварительную проверку меньше ожидаемой экономии. Пилотные измерения $\rho_0, \rho_1, C_{\text{check}}^S, C_{\text{rerun}}^L$ приведены в Е4 (§7.4).

6 Система метрик

6.1 Основные метрики и обоснование формы агрегации

Основными количественными характеристиками работы агента в настоящем исследовании выступают токенная эффективность U_{tok} , денежная эффективность $Q_{\$}$ и доля полностью проверенных решений VCR (Verified Candidate Rate). Приведём сначала определения, затем — расшифровку всех входящих величин.

$$U_{\text{tok}} = \frac{Q}{T}, \quad (17)$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \quad (18)$$

$$\text{VCR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \quad (19)$$

Расшифровка (17)–(19).

Q — агрегированная скалярная оценка качества решения задачи, принимающая значения на $[0, 1]$. По построению Q совпадает с ожидаемым значением функции успеха $\mathbb{E}[\mathcal{V}(Y)]$ (см. (2)), а в случае неоднородного датасета — с взвешенной суммой по классам задач $Q = \sum_b w_b Q_b$, где $Q_b \in [0, 1]$ — средняя успешность по классу b (например, «короткие задачи HumanEval», «длинный контекст SWE-bench»), а $w_b \geq 0$ — доля класса в оцениваемой выборке при $\sum_b w_b = 1$.

T — средний токeнный расход на одну задачу (в токeнах); $U_{\text{tok}} = Q/T$ — «качество, получаемое с 1000 токeнов», то есть показатель токeнной эффективности.

C_{inf} — средняя денежная стоимость инференса на одну задачу (см. (14)); $Q_{\$} = Q/C_{\text{inf}}$ — «качество, получаемое на 1 доллар», то есть показатель денежной эффективности.

N — число задач в оцениваемой выборке.

$\text{tests}_i \in \{0, 1\}$ — результат прогона автоматического тестового набора по i -й задаче: 1, если *все* тесты пройдены, 0 иначе.

$\text{static}_i \in \{0, 1\}$ — результат статического анализа (линтера/типизатора): 1, если не сообщено ни одной ошибки из набора критических правил, 0 иначе.

$\text{sec}_i \in \{0, 1\}$ — результат проверки безопасности (например, Bandit / Semgrep для Python, CodeQL для общих целей): 1, если не обнаружено ни одной уязвимости, 0 иначе.

$\mathbb{I}[\cdot]$ — индикаторная функция: равна 1, если указанное в скобках условие истинно, и 0 иначе. В частности, $\mathbb{I}[\text{static}_i = 1]$ принимает значение 1, если статический анализатор для i -й задачи не выявил критических замечаний, и 0 в противном случае.

$\wedge$ — логическая конъюнкция («и»): выражение $\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1$ истинно лишь тогда, когда *одновременно* выполнены все три условия.

Содержательный смысл VCR. Формула (19) считает *долю* задач, на которых выход агента одновременно прошёл все три класса объективных проверок — функциональные тесты, статический анализ и проверку безопасности. Решение засчитывается только при успехе по всем трём критериям: индикатор в сумме даёт 1, если и только если все три условия истинны, и 0 иначе. Множитель $\frac{1}{N}$ нормирует сумму на число задач N , переводя её в долю от 0 до 1 (в процентах — от 0% до 100%). Таким образом, запись $\frac{1}{N} \sum_i \mathbb{I}[\dots]$ — это обычная *средняя доля успехов*, а не произвольный коэффициент $1/3$: тройки условий здесь нет, есть N задач и усреднение по ним.

Почему именно три критерия.

Три компонента индикатора — тесты, статический анализ и проверка безопасности — задают *минимально достаточный* набор объективных сигналов для патча в инженерной постановке. Функциональные тесты фиксируют корректность, статический анализ — отсутствие явных ошибок и нарушений стиля, проверка безопасности — отсутствие уязвимостей. Конъюнкция консервативна: частичный успех (например, «тесты пройдены, но есть уязвимость») не засчитывается, что соответствует практикам промышленного CI/CD.

6.2 Агрегированная метрика успешности и её форма

Для агрегированной оценки успешности используется форма

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^\lambda, \quad \lambda \in [0, 1], \quad \lambda = 0.5. \quad (20)$$

Обоснование этой конструкции состоит в следующем. Во-первых, величины *Success* и *Verified* являются необходимыми условиями засчитывания решения: patch, не прошедший внешнюю проверку, не должен получать высокий итоговый балл. Во-вторых, множитель M_i^λ моделирует влияние поддерживаемости на итоговую оценку; параметр λ регулирует силу этого влияния. В базовой конфигурации используется $\lambda = 0.5$; анализ E2d обнаружил семь инверсий ранжирования при изменении λ и весов, поэтому составная метрика рассматривается только как вспомогательная.

6.3 Веса поддерживаемости и их обоснование

$$M_i = w_{\text{cc}}(1 - \widehat{\text{CC}}_i) + w_{\text{dup}}(1 - \widehat{\text{Dup}}_i) + w_{\text{warn}}(1 - \widehat{\text{Warn}}_i) + w_{\text{doc}}\widehat{\text{Doc}}_i. \quad (21)$$

Таблица 2 Веса поддерживаемости и их связь с подкатегориями ISO/IEC 25010:2023 [26]

Параметр	Значение	Обоснование
w_{cc}	0.35	ISO 25010 Modifiability; цикломатическая сложность [27] — наиболее прямой прокси стоимости сопровождения.
w_{dup}	0.25	ISO 25010 Reusability; дублирование — ключевой сигнал технического долга.
w_{warn}	0.20	ISO 25010 Analysability; warnings — поверхностный, но надёжный сигнал.
w_{doc}	0.20	ISO 25010 Modifiability; покрытие документацией как прокси понятности.

Анализ чувствительности

В E2d проверяется устойчивость ранжирования конфигураций B0–B3 при вариации $(w_{\text{cc}}, w_{\text{dup}}, w_{\text{warn}}, w_{\text{doc}})$ в $\pm 20\%$ и при $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ в (20). Инверсия ранжирования при таких вариациях рассматривается как признак чувствительности метрики к выбранной параметризации.

6.4 Метрики надёжности

$$\text{SCR} = \frac{\#\{\text{ошибочные кандидаты, выявленные процедурой самопроверки}\}}{\#\{\text{все ошибочные кандидаты}\}}, \quad (22)$$

$$\text{RFR} = \frac{\#\{\text{патчи без отката}\}}{\#\{\text{применённые патчи}\}}. \quad (23)$$

7 Эмпирическая валидация

7.1 Воспроизводимость, исходные данные и статистический протокол

Исследовательский стенд опубликован в репозитории INoT_Research [30]; все приведённые ниже результаты привязаны к commit 010211ee5775185e8330ab6cde06e912c2adcb9e. Перед повторным запуском выполнены встроенная проверка конфигурации и тесты: `python -m inot check` завершилась без ошибок, а `pytest` дал 15 успешных тестов. Используемые в статье исходные JSON находятся в `results/slide_e2e3_real`, `results/e6` и `reproducibility/results/20260726_flash_lite`; их SHA-256, агрегаты и код построения рисунков сохранены в `reproducibility/evidence_snapshot.json`.

Основной набор данных — канонический HumanEval [18]. Про-пилот использует задачи HumanEval/0–4, а повторный Flash-Lite-прогон — HumanEval/0–19 в исходном порядке. Исторический контекст строится детерминированно (seed 42) из постановок и эталонных решений *других* задач HumanEval; эталон целевой задачи не добавляется. Блоки присоединяются целиком, поэтому фактическая длина может превышать целевую. Для $n = 20$ при целевых $\{256, 1024, 4096, 16384\}$ фактические средние длины равны соответственно $\{371.2, 1154.8, 4210.6, 16524.3\}$ токена, а диапазоны — $\{259\text{--}545, 1024\text{--}1411, 4100\text{--}4415, 16389\text{--}16946\}$. Такой контекст воспроизводим, но синтетичен и не моделирует историю реального репозитория во всей полноте.

Сравниваются B2 Classical-MAS (planner, worker и critic как отдельные API-вызовы с повторной передачей контекста) и B3 Hybrid-INoT (внутренний ролевой цикл над однократно переданным контекстом). Про-пилот выполнен через `google/gemini-3.1-pro-preview`, повторный прогон — через `google/gemini-3.1-flash-lite`. В конфигурации записаны цены \$2/\$12 и \$0.25/\$1.50 за миллион входных/выходных токенов соответственно [31, 32]; дата фиксации — 26 июля 2026 г. Эти имена являются API-псевдонимами, а не неизменяемыми снимками весов, поэтому возможен дрейф провайдера.

Для каждой длины контекста образуется пара B2–B3 на одной задаче и одном seed. Используются

$$S_T = 100 \left(1 - \frac{\bar{T}_{B3}}{\bar{T}_{B2}} \right), \quad S_C = 100 \left(1 - \frac{\bar{C}_{B3}}{\bar{C}_{B2}} \right), \quad (24)$$

$$\Delta U_{\text{tok}} = 100 \left(\frac{U_{\text{tok}}^{B3}}{U_{\text{tok}}^{B2}} - 1 \right). \quad (25)$$

Для непрерывной парной разности применяется двусторонний критерий Вилкоксона, для бинарного исхода — точный критерий Мак-Немара; доверительные интервалы получены percentile bootstrap с 10^4 ресэмплирований. Четыре сравнения по длинам контекста корректируются методом Холма–Бонферрони.

7.2 E3 — Масштабирование по длине контекста

Таблица 3 содержит непосредственно наблюдаемые средние. В обоих прогонах расход B2 растёт почти линейно с добавляемым контекстом, тогда как B3 после 4096 целевых токенов остаётся около 5.6–6.7 тыс. токенов вследствие компрессии.

Таблица 3 Реальные API-измерения B2 и B3 на HumanEval

Модель	Контекст	T_{B2}	T_{B3}	S_T , %	S_C , %	pass@1 B2/B3
Pro ($n = 5$)	256	4964	2203	55.6	43.1	1.00/1.00
	1024	7117	2925	58.9	46.8	1.00/1.00
	4096	18218	6706	63.2	49.1	1.00/1.00
	16384	59631	6608	88.9	79.0	1.00/1.00
Flash-Lite ($n = 20$)	256	2738	1379	49.6	26.0	1.00/1.00
	1024	5487	2271	58.6	41.8	1.00/1.00
	4096	16007	5664	64.6	56.9	1.00/1.00
	16384	58110	5633	90.3	86.9	1.00/1.00

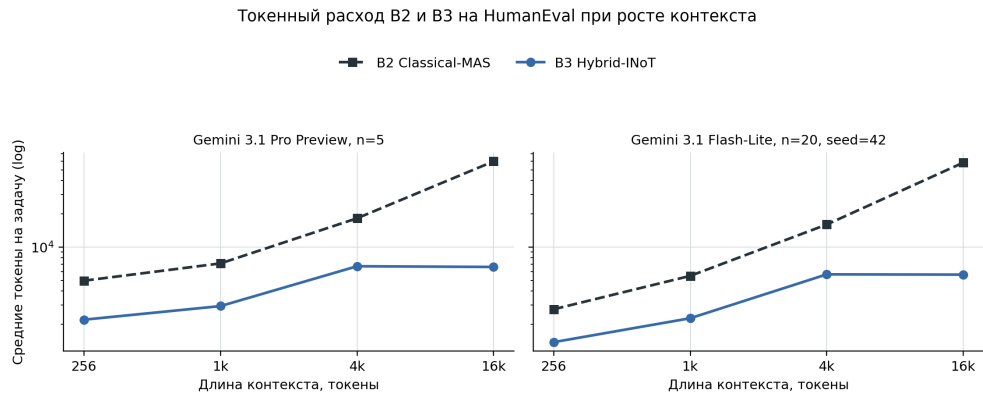


Рис. 1 Средний суммарный токенный расход на задачу. Штриховая линия с квадратами — B2 Classical-MAS; сплошная линия с кругами — B3 Hybrid-INoT. Ось y логарифмическая. Слева показан сохранённый Pro-пилот ($n = 5$ на точку), справа — выполненная для статьи репликация Flash-Lite ($n = 20$, seed 42).

Покажем вычисление для Pro при 16 384 целевых токенах:

$$S_T = 100 \left(1 - \frac{6608.4}{59631.4} \right) = 88.9\%, \quad S_C = 100 \left(1 - \frac{0.0294908}{0.1407228} \right) = 79.0\%.$$

Поскольку pass@1 обеих конфигураций равен 1, агрегированное отношение средних даёт

$$U_{B2} = \frac{1}{59631.4} = 1.677 \cdot 10^{-5}, \quad U_{B3} = \frac{1}{6608.4} = 1.513 \cdot 10^{-4},$$

то есть рост U_{tok} равен 802.4%. При корректном попарном вычислении сначала по каждой задаче, а затем по выборке средний рост составляет 805.7% (95% bootstrap CI 753.4–850.1%).

В Про-пилоте попарные выигрыши U_{tok} положительны на всех длинах, но минимально достижимое двустороннее p при пяти однонаправленных ненулевых парах равно 0.0625; после поправки ни одно сравнение не значимо. В Flash-Lite-прогоне средний попарный рост равен 105.5, 144.3, 183.2 и 934.6%, соответствующие 95% CI не пересекают ноль, а $p = 1.91 \cdot 10^{-6}$ для каждой длины; все четыре сравнения сохраняют значимость после поправки Холма.

Однако наблюдение $\text{pass}@1 = 1.0$ для обеих архитектур является потолочным эффектом. Отсутствуют дискордантные пары, точный p Мак-Немара равен 1, а выборки недостаточны для формального доказательства неухудшения в пределах 2 п.п. Следовательно, подтверждён только токенный компонент Н1. В3 доминирует уже в самой левой точке сетки, поэтому результат скрипта `tau_star=256` следует интерпретировать как левое цензурирование $\tau^* \leq 256$ целевых токенов, а не как точную оценку порога.

7.3 E2 — Абляции, критик и параллелизм

Абляционный Про-пилот содержит пять задач. $\text{pass}@1$ равен 1.0 для полной В3 и вариантов без компрессии, критика и самопроверки. Попарные разности агрегированной успешности для В3 против этих трёх вариантов равны -0.0037 , -0.0262 и 0.0000 , с $p = 1.0$, 0.5 и 1.0 ; ни одна разность не значима после поправки Холма. Доли токенового расхода компонентов составляют 36.6% для компрессии, 29.7% для критика и 33.8% для самопроверки, но это *накладные расходы*, а не доказанный независимый вклад в качество. На подмножестве ошибок В0 оказалось только две задачи; наблюдаемая доля ошибочных одобрений критика равна нулю, что недостаточно для вывода о надёжности критика.

В E2c среднее время В2 равно 19.28 с, В3 — 9.76 с; относительное изменение В3 составляет -49.4% , 95% bootstrap CI разности В3–В2 равен $[-13.05; -6.83]$ с, но критерий Вилкоксона при $n = 5$ даёт $p = 0.0625$. Отдельный детерминированный микротест инструментов действительно показывает, что последовательное выполнение медленнее параллельного на 148.8%, однако прямое сравнение архитектур не показало ожидаемого проигрыша В3. Поэтому Н3 в данном пилоте *не поддерживается*; микротест подтверждает лишь механизм, который предстоит изолировать на задачах с реальными независимыми инструментами. В E2d получено семь инверсий ранжирования при вариации весов и λ ; канонически первым оказался вариант без критика. Это указывает на неустойчивость составной метрики и исключает сильный вывод о необходимости каждого компонента.

7.4 E4 — Экономическая проверка малой модели

Для проверки (16) выполнены по 20 API-случаев HumanEval и приближённого SWE-bench Lite. Большая модель — Gemini 3.1 Pro Preview, малая —

Gemini 3.1 Flash-Lite. Чистая маржа на задачу вычисляется как

$$M = (\rho_0 - \rho_1)C_{\text{rerun}}^L - C_{\text{check}}^S. \quad (26)$$

Таблица 4 Проверка условия окупаемости малой модели, долл. США на задачу

Набор	ρ_0	ρ_1	C_{check}^S	C_{rerun}^L	M (95% CI)	H2
HumanEval	.35	.00	.000060	.010822	.003728 [.002042; .007685]	да
SWE-bench Lite*	.75	.60	.000122	.014469	.002048 [−.000104; .004650]	нет

Для HumanEval, например,

$$M = (0.35 - 0) \cdot 0.0108224 - 0.000060025 = 0.003727815.$$

Нижняя граница CI положительна. Для SWE-bench Lite интервал пересекает ноль, поэтому исходная H2, требующая выполнения условия на обоих наборах, не подтверждена. Кроме того, звёздочка существенна: текущий загрузчик не запускает официальный контейнерный SWE-bench harness [16], а использует извлечение кода и заглушку `check(_)`. Эти результаты характеризуют процедуру маршрутизации/самопроверки стенда, но не способность решать реальные GitHub issues.

7.5 E6 — Межмодельная чувствительность

Сохранённый E6 сравнивает Pro и Flash-Lite на контролируемом наборе из десяти случаев для каждой пары архитектура–контекст. Набор содержит повторения трёх семейств функций (`reverse_words`, `count_peaks`, `normalize_path`); `pass@1` равен 1.0 во всех ячейках, поэтому эксперимент измеряет стоимость на простом потолочном наборе. При длинах 512, 2048 и 8192 стоимость Flash-Lite составляет 4.8, 6.9 и 9.7% стоимости Pro для B2 и 4.4, 5.4 и 7.0% для B3 (рис. 2). Парные различия U_{tok} значимы ($p = 0.001953$; все сравнения проходят поправку Холма), но из-за потолка качества этот результат нельзя обобщать на сложные задачи.

7.6 E5 — Экстраполяция стоимости API-инференса

Полный ТСО-эксперимент не был завершён: дополнительный запуск был остановлен лимитом провайдера до сохранения `runs.json` и исключён из анализа. Вместо подмены отсутствующих данных выполнена явно ограниченная аналитическая экстраполяция из двадцати парных Pro-случаев E3 (пять задач на каждую из четырёх длин, равные веса). Для N задач

$$\Delta C_{\text{inf}}(N) = N \cdot \frac{1}{20} \sum_{i=1}^{20} (C_{B2,i} - C_{B3,i}). \quad (27)$$

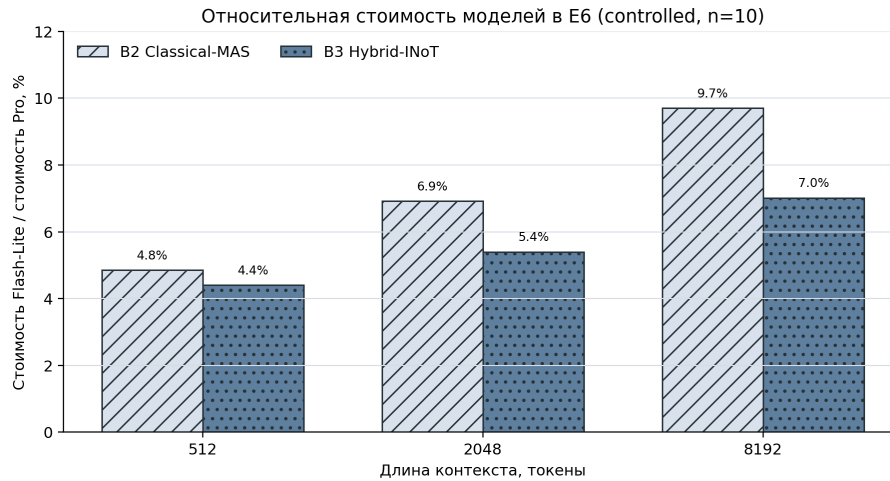


Рис. 2 Отношение средней стоимости Gemini 3.1 Flash-Lite к стоимости Gemini 3.1 Pro Preview в контролируемом E6. Штриховки обеспечивают различимость при чёрно-белой печати.

Средняя разность равна \$0.042996 на задачу; при $N = 10\,000$ это \$429.96 экономии B3 относительно B2. 95% парный bootstrap CI составляет \$267.06–\$617.17. При линейном изменении обеих цен токенов на $-50, -25, 0, +25, +50\%$ оценка равна соответственно \$214.98, \$322.47, \$429.96, \$537.45 и \$644.94 (рис. 3).

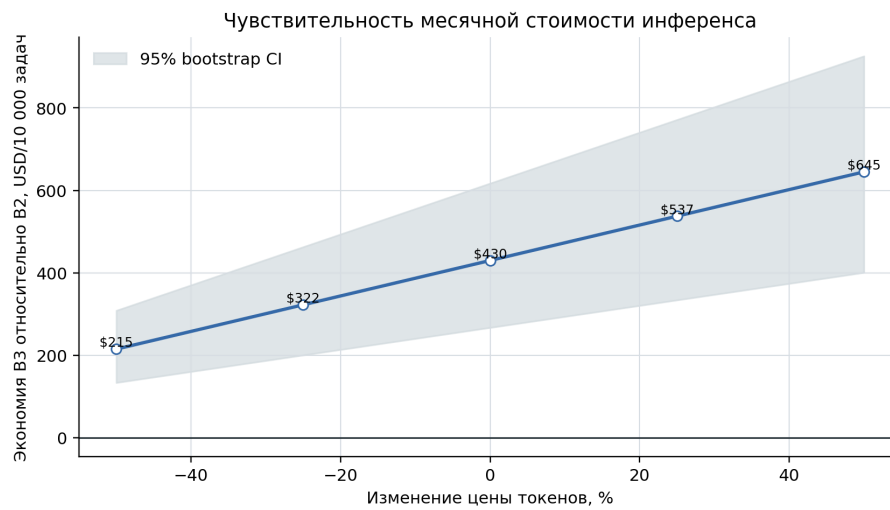


Рис. 3 Чувствительность месячной экономии B3 относительно B2 к общему множителю цены токенов; объём 10 000 задач, равномерная смесь четырёх длин контекста. Полоса — 95% парный bootstrap CI.

8 Экономическая модель

8.1 Граница между инференсом и ТСО

$$\text{ТСО}_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \quad (28)$$

В работе измерено только C_{inf} для внешнего API. Аппаратные ресурсы, энергия, хранение, сопровождение, разработка и возможная стоимость задержки не измерялись; следовательно, положительная нижняя граница в E5 доказывает экономию *API-инференса в пилотной смеси*, но не $\Delta\text{ТСО} < 0$. При использовании управляемого API часть остальных слагаемых включена в цену провайдера непрозрачно, что также не позволяет декомпонировать ТСО.

8.2 Сводный вердикт по гипотезам

Таблица 5 Соответствие исходных гипотез фактическим данным

Гипотеза	Вердикт	Основание
H1	частично	Токенный выигрыш B3 подтверждён в Flash-Lite; Pto недоволен, а неухудшение pass@1 на 2 п.п. не доказано из-за потолка.
H2	не подтверждена в исходном объёме	Положительный CI на HumanEval, но CI пересекает ноль на приближённом SWE-bench Lite.
H3	не поддержана	Инструментальный микротест показывает пользу параллелизма, но B3 в прямом пилоте быстрее B2; $n = 5$.
τ^*	не идентифицирован	B3 доминирует в левой точке; установлено только $\tau^* \leq 256$ целевых токенов на этой сетке.

9 Ограничения и угрозы валидности

9.1 Границы интерпретации

- Малые и неслучайные выборки.** Использованы первые 5 или 20 задач HumanEval, а не случайная стратифицированная выборка из 164 задач; новый прогон имеет один seed. Это ограничивает статистическую мощьность и внешнюю валидность.
- Синтетический контекст.** Длинный вход составлен из других задач HumanEval, а не из истории реального репозитория. Он изолирует цену повторной передачи, но не отражает зависимости и шум промышленной кодовой базы.
- Потолок качества.** В B2/B3 и контролируемом E6 pass@1 равен 1.0. Данные показывают различие расхода, но не дают строгого теста неухудшения качества и не исключают иной результат на сложных задачах.
- Неполный SWE-bench.** Используемая проверка не является официальным контейнерным протоколом; вывод о решении issue или переносе на реальные репозитории не делается.

5. **Дрейф API.** В результатах сохранены имена моделей, параметры, токены, задержка и стоимость, но API-псевдонимы не гарантируют неизменность весов. Неудавшиеся из-за лимита провайдера запуски не включены.
6. **Ограниченная аудируемость ответа.** `runs.json` хранит метрики и `usage`, но не полный финальный текст решения; независимая постфактум-проверка каждого ответа ограничена.
7. **Стоимость не равна ТСО.** Рисунок 3 отражает только API-инференс при заданной смеси контекстов. Остальные слагаемые (28) и производственная нагрузка не измерялись.
8. **Архитектурные границы.** Истинно независимые подзадачи, необходимость процессной изоляции, ошибки критика и потеря фактов при компрессии могут устранить преимущество В3. Промежуточные ролевые тексты являются вычислительным механизмом, а не достоверным объяснением поведения модели [9].

10 Направления дальнейшего исследования

Следующий этап должен устранять выявленные источники неопределённости, а не повторять уже выполненный пилот:

1. провести HumanEval-эксперимент на всех 164 задачах минимум с пятью seed и заранее зафиксированным планом анализа; дополнить pass@1 тестом неухудшения с требуемой мощностью;
2. расширить сетку контекста ниже 256 токенов, чтобы оценить, а не только цензурировать τ^* ;
3. сохранять полный вход, выход, хеш промпта, точный идентификатор снимка модели и результат внешнего теста для каждого запуска;
4. заменить приближённый SWE-bench-путь официальным контейнерным harness и провести оценку на случайной выборке репозиторий;
5. повторить E2c на задачах с независимыми реальными инструментами и сопоставить последовательный и параллельный режимы при одинаковой модели;
6. сравнить В3 не только с В2, но и с Self-Refine [7], Reflexion [8], ReAct [6] и SWE-agent [17];
7. измерить полный ТСО, включая труд сопровождения, задержку, число повторных запусков, инфраструктуру и хранение, на производственном распределении длины контекста.

11 Заключение

Hybrid-INoT формализован как промежуточная архитектура между внутриконтекстной самокритикой и внешней мультиагентной оркестрацией. Аналитическая модель предсказывает рост преимущества по мере удлинения общего контекста; два реальных API-пилота согласуются с этим предсказанием. В репликации Flash-Lite на 20 задачах В3 использовал на 49.6–90.3% меньше токенов, чем В2, а попарный выигрыш U_{tok} значим после поправки на множественные сравнения. Вместе с тем потолочный pass@1 не позволяет доказать неухудшение качества, Н2

поддержана только на HumanEval, НЗ в прямом пилоте не поддерживается, а расчёт \$429.96 на 10 000 задач относится только к API-инференсу. Поэтому обоснованный вывод статьи имеет более узкий охват, чем исходный тезис: Hybrid-INoT является перспективным способом уменьшения повторной передачи длинного контекста, но его качество, порог применимости и полный экономический эффект требуют более мощного исследования на реальных репозиториях.

References

- [1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.
- [2] A. J. Ko, R. DeLine, and G. Venolia, “Information Needs in Collocated Software Development Teams,” in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.
- [3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.
- [4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, “Collaboration, Information Seeking and Communication,” *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.
- [5] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.
- [6] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” *ICLR*, 2023.
- [7] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” *NeurIPS*, 2023.
- [8] N. Shinn *et al.*, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting,” *NeurIPS*, 2023.
- [10] G. Weiss, *Multiagent Systems*. MIT Press, 1999.
- [11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [12] S. Hong *et al.*, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” *ICLR*, 2024.
- [13] C. Qian *et al.*, “ChatDev: Communicative Agents for Software Development,” *ACL*, 2024.

- [14] T. Guo *et al.*, “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” arXiv:2402.01680, 2024.
- [15] C. Qian *et al.*, “Scaling Large Language Model-based Multi-Agent Collaboration,” arXiv:2406.07155, 2024.
- [16] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *ICLR*, 2024.
- [17] J. Yang *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *NeurIPS*, 2024.
- [18] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [19] J. Austin *et al.*, “Program Synthesis with Large Language Models,” arXiv:2108.07732, 2021.
- [20] H. Jiang *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL*, 2024.
- [21] Z. Pan *et al.*, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” *ACL Findings*, 2024.
- [22] H. Jia, E. T. Barr, and S. Mechtaev, “Compressing Code Context for LLM-based Issue Resolution,” arXiv:2603.28119, 2026.
- [23] Z. Bi *et al.*, “Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback,” arXiv:2403.16792, 2024.
- [24] C.-Y. Su and C. McMillan, “Do Code LLMs Do Static Analysis?,” arXiv:2505.12118, 2025.
- [25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, “Enhancing LLM-Based Code Generation with Complexity Metrics,” arXiv:2505.23953, 2025.
- [26] ISO/IEC 25010:2023, “Systems and software engineering — SQuaRE — Product quality model,” ISO, 2023.
- [27] T. J. McCabe, “A Complexity Measure,” *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [28] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [29] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian J. Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

- [30] D. Privezentsev, “INoT_Research: reproducible experiments for Hybrid-INoT,” GitHub repository, commit 010211ee5775185e8330ab6cde06e912c2adcb9e, 2026. [Online]. Available: https://github.com/daniil-novel/INoT_Research. Accessed: 26 July 2026.
- [31] OpenRouter, “Gemini 3.1 Pro Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-pro-preview/pricing>. Accessed: 26 July 2026.
- [32] OpenRouter, “Gemini 3.1 Flash Lite Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-flash-lite/pricing>. Accessed: 26 July 2026.